



# 長安大學

## 数字逻辑实验课 实验报告

课程名称: 数字逻辑实验课

实验名称: VGA 接口

专业名称: 计算机科学与技术/卓越工程师

小组成员: xxx

指导教师: 马峻岩

---

## 一、实验目的

1. 学习并掌握运用 Verilog 语言进行交通灯控制信号设计；
2. 学习并掌握 VGA DB15 接口及其信号定义；
3. 掌握 VGA 信号接口的时序定义；
4. 通过 FPGA 设计 VGA 显示控制程序。

主要仪器和设备：计算机，Basys3 开发板。

## 二、实验要求

电路设计规格描述如下：

1. VGA 控制器产生的 VGA 信号规格：640 × 480，刷新频率为 60Hz。
2. VGA 控制器默认在屏幕中间通过色块显示自己的 3 位组号，颜色和样式自定。
3. 输入信号: sw0，控制显示的数字是否移动，sw=0 时，数字静止，sw=1 时数字向右移动，当达到屏幕最右端时会逐渐穿越到屏幕最左端。
4. 自主进行设计扩展。

## 三、相关外设接口及原理

### 1. Basys3 VGA 接口概述及信号定义

Basys 3 板使用 14 个 FPGA 信号创建了一个 VGA 端口，该端口具有每种颜色 4 位和两个标准同步信号（HS – 行同步/水平同步和 VS – 场同步/垂直同步）。颜色信号使用电阻分压电路，这些电路与 VGA 显示器的 75 欧姆终端电阻相配合，在红色、绿色和蓝色 VGA 信号上分别创建 16 个信号电平。这个接口信号定义和电路原理如图 3-1 所示，生成的视频颜色信号在 0V（完全关闭）和 0.7V（完全打开）之间以相等的增量变化。注意 VGA 接口的 R、G、B 为模拟信号，对每种颜色 FPGA 使用 4 个数字信号的和电阻网络构成了简单的数模转换电路。使用这个电路，可以显示 4096（ $12^2$ ）种不同的颜色，每种颜色对应一个独特的 12 位模式。必须在 FPGA 中创建一个视频控制器电路，以正确的时序驱动同步和颜色信号，从而生成一个工作的显示系统。

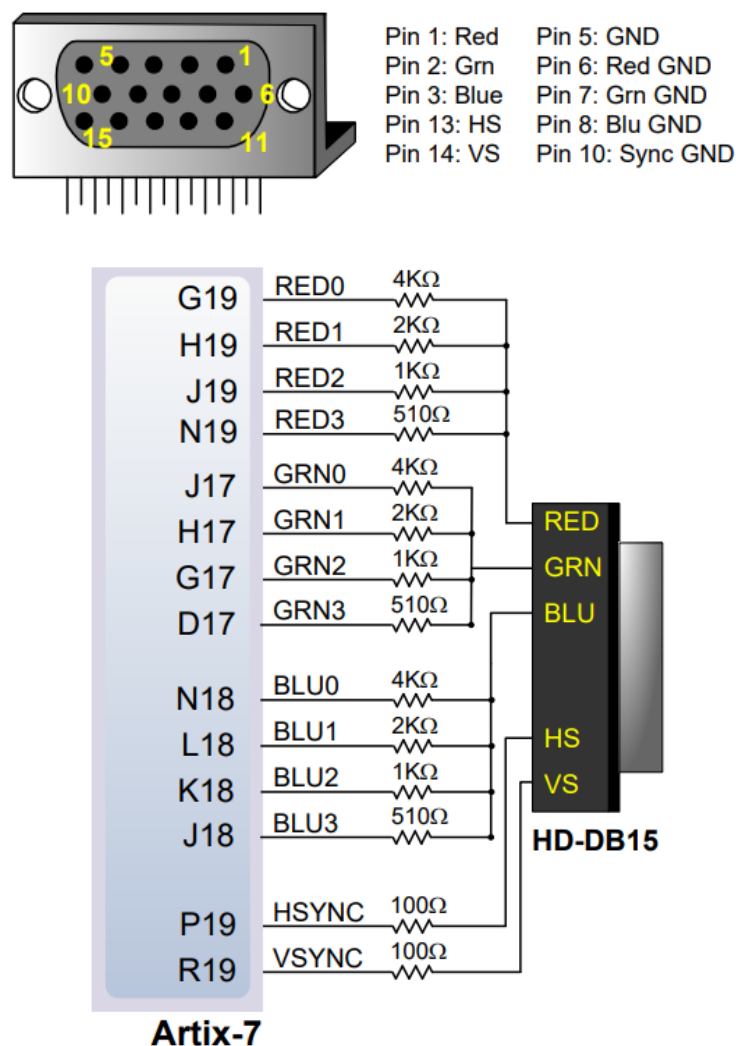


图 3-1 VGA 信号定义和电路原理图

## 2. VGA 工作原理

VGA 信号时序由 VESA® 组织 ([www.vesa.org](http://www.vesa.org)) 指定、发布、版权保护并销售。以下 VGA 系统时序信息作为示例,说明如何在 640 x 480 模式下驱动 VGA 显示器。有关更详细的信息,或有关其他 VGA 频率的信息,请参阅 VESA 网站上的文档。

VGA 早期用于阴极射线管 (Cathode Ray Tube, 简称 CRT) 显示器接口, CRT 显示器基于电子束扫描和荧光物质发光。电子枪发射电子束,经聚焦、偏转后扫描屏幕,撞击荧光粉发出光子,形成图像。每秒数十至数百次扫描,呈现连续图像。虽然 CRT 显示器已经不常见了,但 VGA 接口还在很多场合下游广泛的应用。

电子束从屏幕的左上角开始,从左向右逐行扫描,每扫描完一行,电子束回到屏幕的左边下一行的起始位置,在这期间, CRT 对电子束进行消隐。每行结束时,用行同步信号进行行同步。当扫描完所有的行,形成一帧,用场同步信号进行场同步,并使

扫描回到屏幕左上方，同时进行场消隐，开始下一帧的扫描，如图 3-2 所示。

视频数据通常来自视频刷新内存；每个像素位置分配一个或多个字节（Basys 3 每个像素使用 12 位）。当光束在显示器上移动时，控制器必须索引视频内存，并在电子光束移动到给定像素时精确地检索并应用视频数据到显示器上。

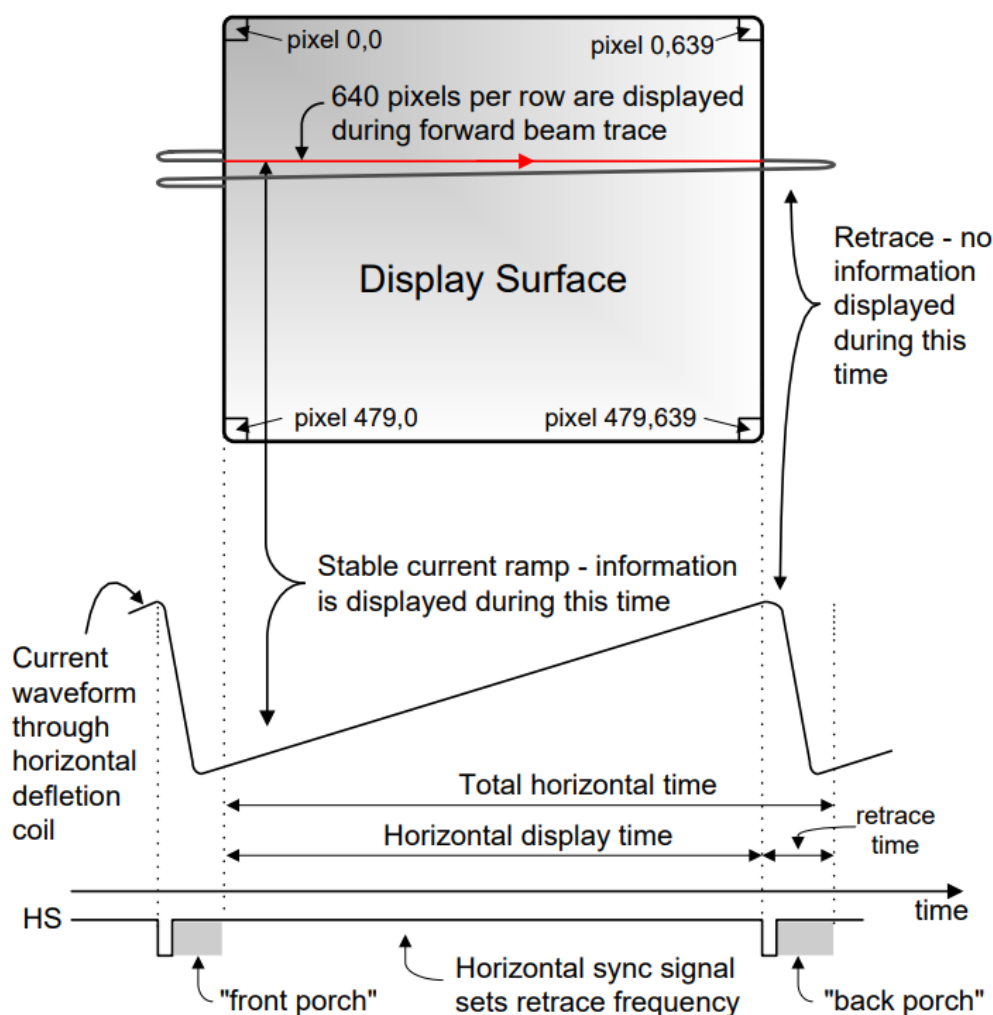
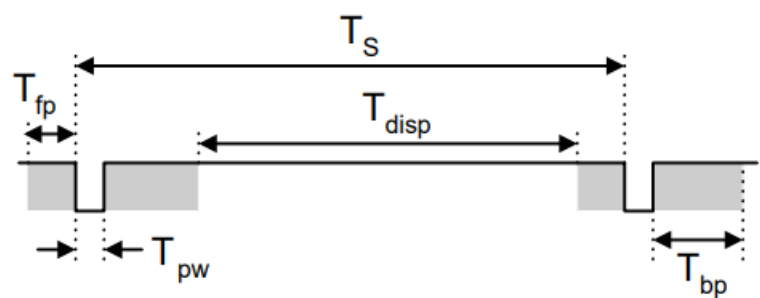


图 3-2 VGA 逐行扫描显示过程

### 3. VGA 一帧时序图

VGA 控制器电路必须生成 HS 和 VS 时序信号，并根据像素时钟协调视频数据的传送。像素时钟定义了显示一个像素信息所需的时间。VS 信号定义了显示器的“刷新”频率，即重绘显示器上所有信息的频率。最低刷新频率取决于显示器的荧光粉和电子束强度，实际刷新频率在 50Hz 到 120Hz 之间。给定刷新频率下显示的行数定义了水平“回扫”频率。对于使用 25 MHz 像素时钟和 60 +/-1Hz 刷新率的 640 像素 x 480 行显示器，可以推导出图 3-3 所示的信号时序。同步脉冲宽度和前后消隐间隔（front porch、back porch 见图 3-2，消隐间隔是在同步脉冲期间和之后无法显示信息

的时间) 的时序基于对实际 VGA 显示器的观察。



| Symbol     | Parameter    | Vertical Sync |         |       | Horiz. Sync |      |
|------------|--------------|---------------|---------|-------|-------------|------|
|            |              | Time          | Clocks  | Lines | Time        | Clks |
| $T_S$      | Sync pulse   | 16.7ms        | 416,800 | 521   | 32 us       | 800  |
| $T_{disp}$ | Display time | 15.36ms       | 384,000 | 480   | 25.6 us     | 640  |
| $T_{pw}$   | Pulse width  | 64 us         | 1,600   | 2     | 3.84 us     | 96   |
| $T_{fp}$   | Front porch  | 320 us        | 8,000   | 10    | 640 ns      | 16   |
| $T_{bp}$   | Back porch   | 928 us        | 23,200  | 29    | 1.92 us     | 48   |

图 3-3 使用 25 MHz 像素时钟和 60 Hz 垂直刷新率的 640×480 行显示器的信号时序。

#### 4. VGA 控制器

VGA 控制器电路，如图 3-4 示，通过解码由像素时钟驱动的水平同步计数器的输出出来生成 HS 信号时序。你可以使用这个计数器来定位给定行上的任意像素位置。同样，随着每个 HS 脉冲递增的垂直同步计数器的输出可以用来生成 VS 信号时序，并且你可以使用这个计数器来定位任意给定的行。这两个不断运行的计数器可以用于形成视频 RAM 的地址。HS 脉冲的开始与 VS 脉冲的开始之间没有指定时间关系，因此你可以安排计数器以便轻松形成视频 RAM 地址，或最小化同步脉冲生成的解码逻辑。

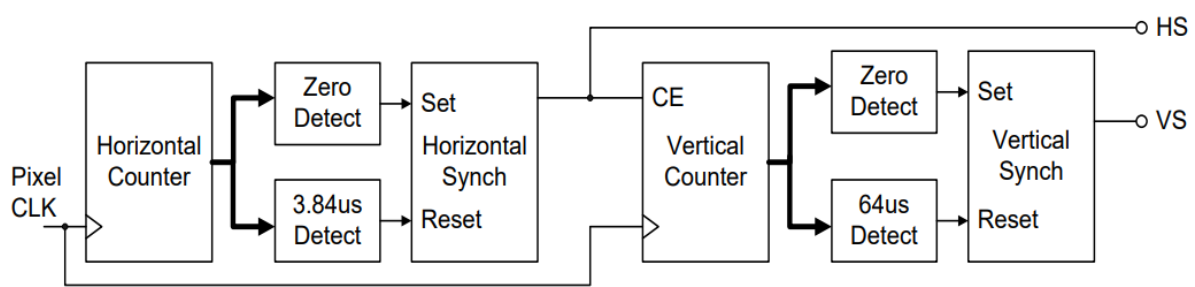


图 3-4 VGA 显示控制器模块图

## 四、实验步骤及关键代码

### 1. 系统模块划分及框图

根据设计规格，本实验包含三个模块，具体模块名，信号说明如表 4-1 所示。

表 4-1 系统主要模块用说明

| 模块名            | 模块主要功能说明                             |
|----------------|--------------------------------------|
| divide_clock   | 100MHz 时钟 4 分频至 25MHz，用于驱动 vga 逐像素扫描 |
| VGA_controller | 控制 VGA 显示，模块输出行扫描信号、场扫描信号和像素点着色信息    |
| vga_top        | 顶层模块，将时钟分频、VGA 显示控制模块封装              |

表 4-2 模块输入、输出信号说明

| 信号接口名称   | 信号所属模块         | 类型<br>(input/output) | 信号说明            |
|----------|----------------|----------------------|-----------------|
| clk      | divide_clock   | input                | 分频的时钟信号         |
| reset    | divide_clock   | input                | 分频的置位信号         |
| divClk   | divide_clock   | output               | 分频结果的输出信号       |
| clk      | VGA_controller | input                | VGA 控制模块的扫描时钟信号 |
| reset    | VGA_controller | input                | VGA 控制模块的置位信号   |
| en_shift | VGA_controller | input                | 移动显示的使能信号       |
| vga_hs   | VGA_controller | output               | 行扫描信号           |
| vga_vs   | VGA_controller | output               | 场扫描信号           |
| vga_rgb  | VGA_controller | output               | 像素点颜色值          |

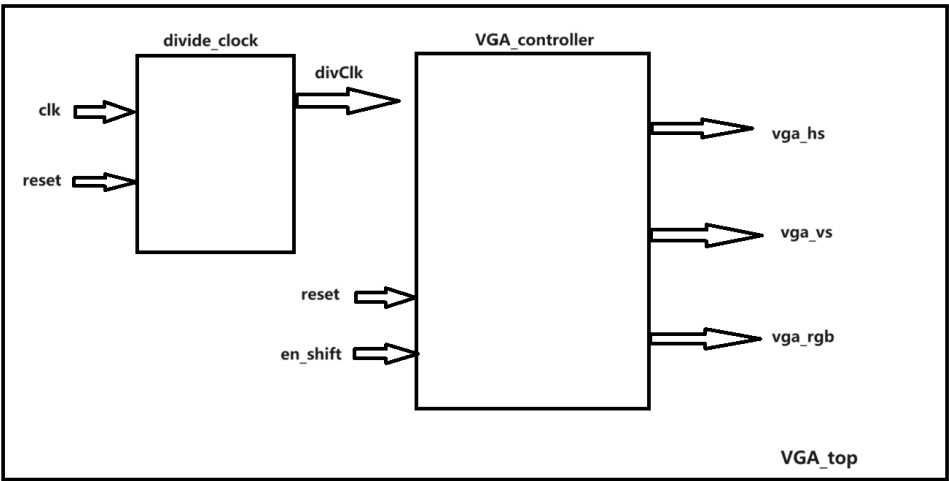


图 4-1 系统模块及连接图（含信号名）

### 2. 控制器设计

## (1) 关键模块设计思路及核心 Verilog 代码

```
1 module divide_clock(clk, reset, divClk);
2     input clk, reset;
3     output reg divClk;
4
5     reg cnt;
6
7     always @(posedge clk, posedge reset)
8     begin
9         if (reset)
10            begin
11                cnt <= 1'b0;
12                divClk <= 1'b0;
13            end
14        else
15            begin
16                if (cnt == 1)
17                    begin
18                        divClk <= ~divClk;
19                        cnt <= 1'b0;
20                    end
21                else
22                    cnt <= cnt + 1;
23            end
24        end
25    endmodule
```

Figure 1 divide\_clock

```
1 module vga_top(clk, sw, vgaRed, vgaGreen, vgaBlue, Hsync, Vsync);
2     input clk;
3     input [1:0] sw;
4
5     output [3:0] vgaRed;
6     output [3:0] vgaGreen;
7     output [3:0] vgaBlue;
8     output Hsync, Vsync;
9
10    wire divClk;
11    divide_clock div_clk(.clk(clk), .reset(sw[0]), .divClk(divClk));
12    VGA_controller vga_contr(.clk(divClk), .reset(sw[0]), .en_shift(sw[1]),
13        .vga_hs(Hsync), .vga_vs(Vsync), .vga_rgb({vgaRed[3:0], vgaGreen[3:0], vgaBlue[3:0]}));
14
15    endmodule
```

Figure 2 VGA\_top

```

1  `define VGA_640x480x60
2
3  module VGA_controller(clk, reset, en_shift, vga_hs, vga_vs, vga_rgb);
4      input clk, reset;
5      input en_shift;
6      output reg vga_hs;
7      output reg vga_vs;
8
9      wire en_hs;
10     wire en_vs;
11     wire en;
12
13     wire [9:0] pos_x;
14     wire [9:0] pos_y;
15
16     output reg [11:0] vga_rgb;
17
18     `ifdef VGA_640x480x60
19         localparam HS_A = 96; // 行脉冲区域
20         localparam HS_B = 48; // 行显示后沿
21         localparam HS_C = 640; // 行有效显示区域
22         localparam HS_D = 16; // 行显示前沿>
23         localparam HS_E = 800; // 行扫描周期
24
25         localparam VS_A = 2; // 场脉冲区域
26         localparam VS_B = 33; // 场显示后沿
27         localparam VS_C = 480; // 场有效显示区域
28         localparam VS_D = 10; // 场显示后沿
29         localparam VS_E = 525; // 场扫描周期
30     `endif
31
32     //NOTE 行计数器, 对像素时钟计数(记录所在行已经扫描到的列数, 计数器满代表一行结束)
33     reg cnt_hs;
34     always @(posedge clk, posedge reset)
35     begin
36         if (reset)
37             cnt_hs <= 0;
38         else
39             begin
40                 if (cnt_hs < HS_E - 1)
41                     cnt_hs <= cnt_hs + 1;
42                 else
43                     cnt_hs <= 0;
44             end
45     end
46
47     //NOTE 场计数器, 对行计数(计数器满代表一帧结束)
48     reg cnt_vs;
49     always @(posedge clk, posedge reset)
50     begin
51         if (reset)
52             cnt_vs <= 0;
53         else
54             begin
55                 if (cnt_hs == HS_E - 1)
56                     begin
57                         if (cnt_vs < VS_E - 1)
58                             cnt_vs <= cnt_vs + 1;
59                         else
60                             cnt_vs <= 0;
61                     end
62                 else
63                     cnt_vs <= cnt_vs;
64             end
65     end
66
67     //NOTE 行同步信号(同步之前 vga_hs 信号为低, 同步之后 vga_hs 信号为高)
68     always @(posedge clk, posedge reset)
69     begin
70         if (reset)
71             vga_hs <= 1'b1;
72         else
73             if (cnt_hs < HS_A)
74                 vga_hs <= 1'b0;
75             else
76                 vga_hs <= 1'b1;
77     end
78
79     //NOTE 场同步信号(同步之前 vga_vs 信号为低, 同步之后 vga_vs 信号为高)
80     always @(posedge clk, posedge reset)
81     begin
82         if (reset)
83             vga_vs <= 1'b1;
84         else
85             if (cnt_vs < VS_A)
86                 vga_vs <= 1'b0;
87             else
88                 vga_vs <= 1'b1;
89     end
90
91     //NOTE 产生使能信号
92     assign en_hs = (cnt_hs >= HS_A + HS_B - 1) && (cnt_hs < HS_E - HS_D);
93     assign en_vs = (cnt_vs >= VS_A + VS_B - 1) && (cnt_vs < VS_E - VS_D);
94     assign en = en_hs && en_vs;
95
96     //NOTE 产生像素点坐标
97     assign pos_x = en ? (cnt_hs - (HS_A + HS_B - 1'b1)) : 0;
98     assign pos_y = en ? (cnt_vs - (VS_A + VS_B - 1'b1)) : 0;
99
100
101     //NOTE 偏移运动时钟
102     reg [24:0] shift_cnt;
103     reg shift_clk;
104     always @(posedge clk, posedge reset)
105     begin
106         if (reset)
107             begin
108                 shift_clk <= 0;
109                 shift_cnt <= 0;
110             end
111         else
112             begin
113                 if (shift_cnt == 25'd24_999_999)
114                     begin
115                         shift_clk <= ~shift_clk;
116                         shift_cnt <= 0;
117                     end
118                 else
119                     shift_cnt <= shift_cnt + 1;
120             end
121     end
122
123     //NOTE 偏移运动标志位的变化
124     reg [9:0] pos0;
125     reg [9:0] pos1;
126     reg [9:0] pos2;
127     always @(posedge shift_clk, posedge reset)
128     begin
129         if (reset)
130             begin
131                 pos0 <= 6;
132                 pos1 <= 16;
133                 pos2 <= 26;
134             end
135         else if (en_shift)
136             begin
137                 pos0 <= (pos0 + 1) % 40;
138                 pos1 <= (pos1 + 1) % 40;
139                 pos2 <= (pos2 + 1) % 40;
140             end
141         else
142             begin
143                 pos0 <= 6;
144                 pos1 <= 16;
145                 pos2 <= 26;
146             end
147     end

```

Figure 3 VGA\_controller(part 1)

```

1 always @(posedge clk, posedge reset)
2 begin
3     if (reset)
4         vga_rgb <= 12'h000;
5     else
6         begin
7             if (pos0 < (pos0 + 7) % 40 && pos1 < (pos1 + 7) % 40 && pos2 < (pos2 + 7) % 40) //NOTE pos0、pos1、pos2 处的数字均未被截断
8                 begin
9                     //NOTE 显示 pos0 处的数字 0
10                    if(pos_x[9:4] >= pos0 && pos_x[9:4] < pos0 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 10) //第 0 段
11                        vga_rgb = 12'h00f;
12                    else if(pos_x[9:4] >= pos0 + 6 && pos_x[9:4] < pos0 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 1 段
13                        vga_rgb = 12'h00f;
14                    else if(pos_x[9:4] >= pos0 + 6 && pos_x[9:4] < pos0 + 8 && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 2 段
15                        vga_rgb = 12'h00f;
16                    else if(pos_x[9:4] >= pos0 && pos_x[9:4] < pos0 + 8 && pos_y[9:4] >= 20 && pos_y[9:4] < 22) //第 3 段
17                        vga_rgb = 12'h00f;
18                    else if(pos_x[9:4] >= pos0 && pos_x[9:4] < pos0 + 2 && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 4 段
19                        vga_rgb = 12'h00f;
20                    else if(pos_x[9:4] >= pos0 && pos_x[9:4] < pos0 + 2 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 5 段
21                        vga_rgb = 12'h00f;
22
23                    //NOTE 显示 pos1 处的数字 0
24                    else if(pos_x[9:4] >= pos1 && pos_x[9:4] < pos1 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 10) //第 0 段
25                        vga_rgb = 12'h00f;
26                    else if(pos_x[9:4] >= pos1 + 6 && pos_x[9:4] < pos1 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 1 段
27                        vga_rgb = 12'h00f;
28                    else if(pos_x[9:4] >= pos1 + 6 && pos_x[9:4] < pos1 + 8 && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 2 段
29                        vga_rgb = 12'h00f;
30                    else if(pos_x[9:4] >= pos1 && pos_x[9:4] < pos1 + 8 && pos_y[9:4] >= 20 && pos_y[9:4] < 22) //第 3 段
31                        vga_rgb = 12'h00f;
32                    else if(pos_x[9:4] >= pos1 && pos_x[9:4] < pos1 + 2 && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 4 段
33                        vga_rgb = 12'h00f;
34                    else if(pos_x[9:4] >= pos1 && pos_x[9:4] < pos1 + 2 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 5 段
35                        vga_rgb = 12'h00f;
36
37                    //NOTE 显示 pos2 处的数字 9
38                    else if (pos_x[9:4] >= pos2 && pos_x[9:4] < pos2 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 10) //第 0 段
39                        vga_rgb = 12'h00f;
40                    else if (pos_x[9:4] >= pos2 + 6 && pos_x[9:4] < pos2 + 8 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 1 段
41                        vga_rgb = 12'h00f;
42                    else if (pos_x[9:4] >= pos2 + 6 && pos_x[9:4] < pos2 + 8 && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 2 段
43                        vga_rgb = 12'h00f;
44                    else if (pos_x[9:4] >= pos2 && pos_x[9:4] < pos2 + 8 && pos_y[9:4] >= 20 && pos_y[9:4] < 22) //第 3 段
45                        vga_rgb = 12'h00f;
46                    else if (pos_x[9:4] >= pos2 && pos_x[9:4] < pos2 + 2 && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 5 段
47                        vga_rgb = 12'h00f;
48                    else if (pos_x[9:4] >= pos2 && pos_x[9:4] < pos2 + 8 && pos_y[9:4] >= 14 && pos_y[9:4] < 16) //第 6 段
49                        vga_rgb = 12'h00f;
50                    else
51                        vga_rgb = 12'h000;
52                end
53            else if (pos0 > (pos0 + 7) % 40 && pos1 < (pos1 + 7) % 40 && pos2 < (pos2 + 7) % 40) //NOTE pos0处的数字被截断
54                begin
55                    //NOTE 显示 pos0 处被截断的数字 0
56                    if(((pos_x[9:4] >= pos0 && pos_x[9:4] < 40) || (pos_x[9:4] >= 0 && pos_x[9:4] <= (pos0 + 7) % 40)) //第 0 段
57                        && pos_y[9:4] >= 8 && pos_y[9:4] < 10)
58                        vga_rgb <= 12'h00f;
59                    else if ((pos0 + 7) % 40 == 0 && (pos_x[9:4] == 39 || pos_x[9:4] == 0) && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 1 段(断离)
60                        vga_rgb = 12'h00f;
61                    else if ((pos0 + 7) % 40 > 0 && pos_x[9:4] >= (pos0 + 6) % 40 && pos_x[9:4] <= (pos0 + 7) % 40 //第 1 段(未断离)
62                        && pos_y[9:4] >= 8 && pos_y[9:4] < 16)
63                        vga_rgb = 12'h00f;
64                    else if((pos0 + 7) % 40 == 0 && (pos_x[9:4] == 39 || pos_x[9:4] == 0) && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 2 段(断离)
65                        vga_rgb = 12'h00f;
66                    else if ((pos0 + 7) % 40 > 0 && pos_x[9:4] >= (pos0 + 6) % 40 && pos_x[9:4] <= (pos0 + 7) % 40 //第 2 段(未断离)
67                        && pos_y[9:4] >= 14 && pos_y[9:4] < 22)
68                        vga_rgb = 12'h00f;
69                    else if(((pos_x[9:4] >= pos0 && pos_x[9:4] < 40) || (pos_x[9:4] >= 0 && pos_x[9:4] <= (pos0 + 7) % 40)) //第 3 段
70                        && pos_y[9:4] >= 20 && pos_y[9:4] < 22)
71                        vga_rgb <= 12'h00f;
72                    else if((pos0 + 1) % 40 == 0 && (pos_x[9:4] == 39 || pos_x[9:4] == 0) && pos_y[9:4] >= 14 && pos_y[9:4] < 22) //第 4 段(断离)
73                        vga_rgb = 12'h00f;
74                    else if ((pos0 + 1) % 40 < 40 && pos_x[9:4] >= pos0 && pos_x[9:4] <= (pos0 + 1) % 40 //第 4 段(未断离)
75                        && pos_y[9:4] >= 14 && pos_y[9:4] < 22)
76                        vga_rgb = 12'h00f;
77                    else if((pos0 + 1) % 40 == 0 && (pos_x[9:4] == 39 || pos_x[9:4] == 0) && pos_y[9:4] >= 8 && pos_y[9:4] < 16) //第 5 段(断离)
78                        vga_rgb = 12'h00f;
79                    else if ((pos0 + 1) % 40 < 40 && pos_x[9:4] >= pos0 && pos_x[9:4] <= (pos0 + 1) % 40 //第 5 段(未断离)
80                        && pos_y[9:4] >= 8 && pos_y[9:4] < 16)
81                        vga_rgb = 12'h00f;

```

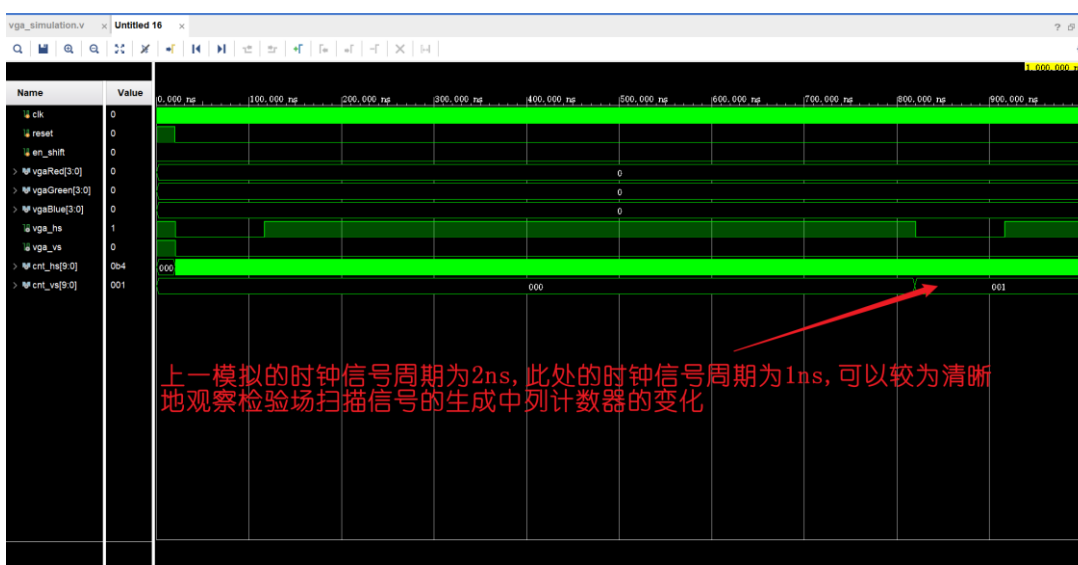
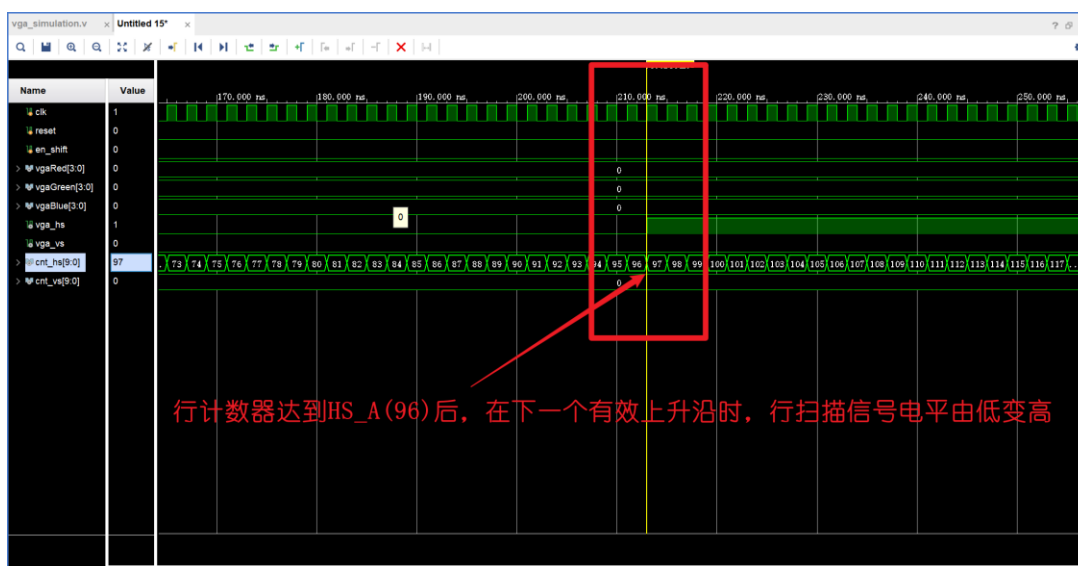
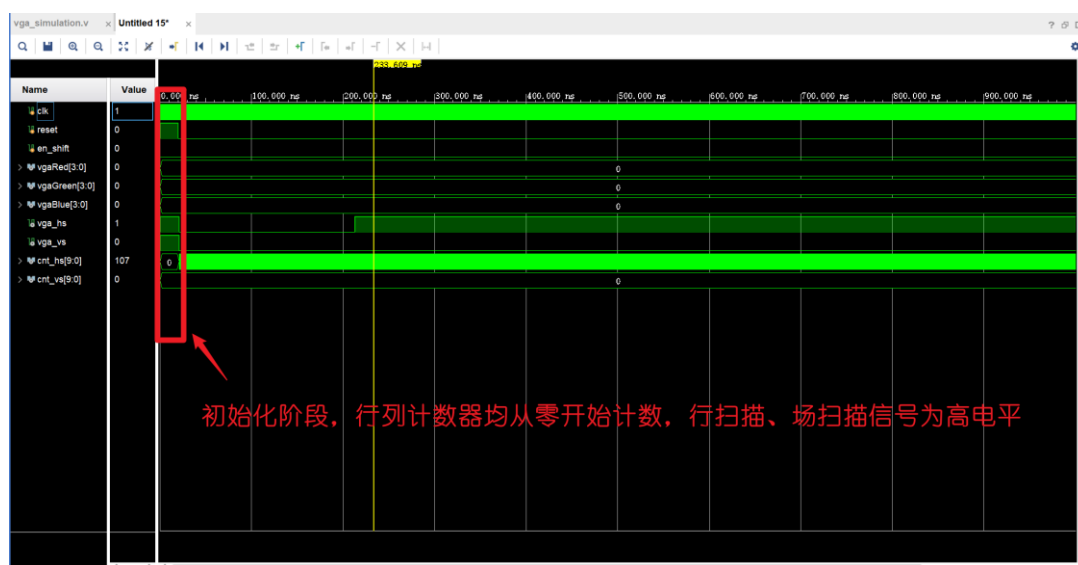
Figure 4 VGA\_controller(part 2)

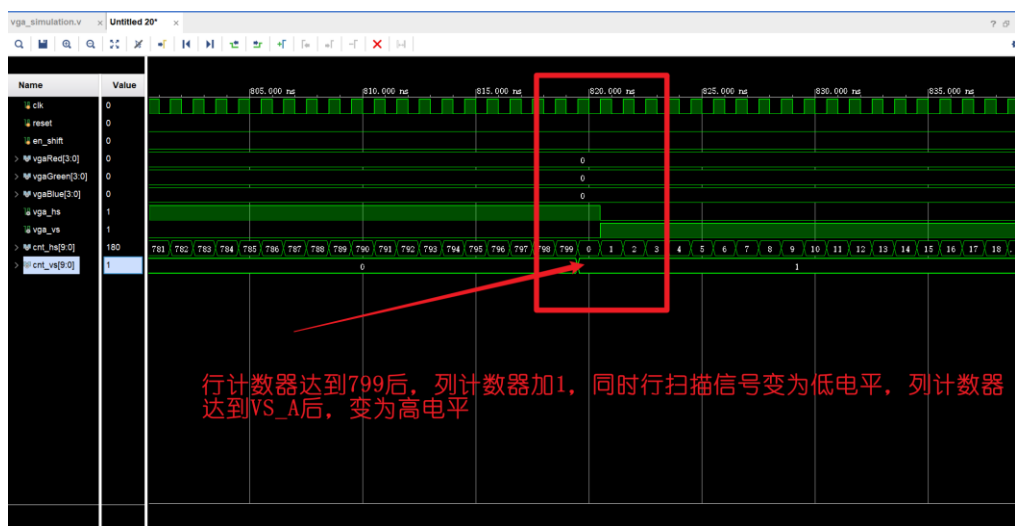
```

end
end
end

```

## (2) 控制器仿真测试





### 3. 开发板测试与验证

表 4-3 功能测试

| 功能点  | 测试方法步骤描述                            | 预期测试结果         | 测试是否通过 |
|------|-------------------------------------|----------------|--------|
| 静态显示 | Reset 信号端置为低电平                      | 屏幕会出现静态的组号 009 | 是      |
| 动态显示 | 保持 reset 信号为低电平，<br>en_shift 信号为高电平 | 屏幕会出现动态的组号 009 | 是      |

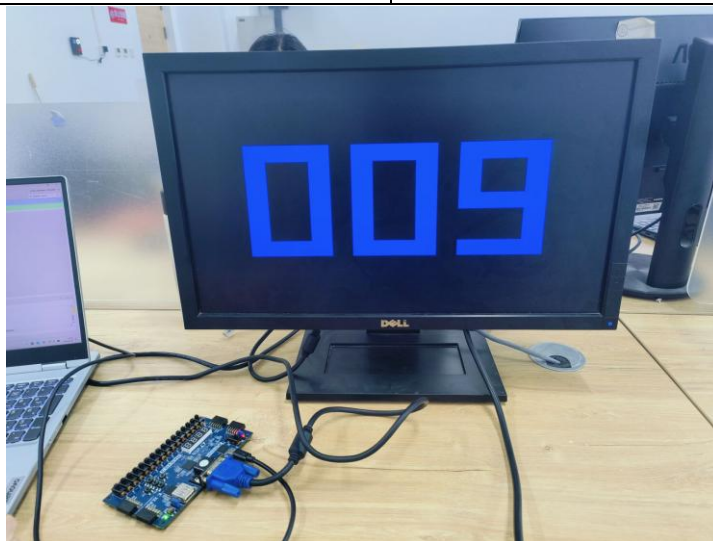


Figure 6 静态显示

动态显示的结果见提交的视频。

## 五、遇到的问题及解决方法

问题：对于动态显示，如何解决数字的连续显示效果。需要对数字的显示位置和具体的数字形态进行分类讨论。

针对数字的移动需求，设置角标，确定位置

将所要显示的3位数字的左上角的坐标的X分量设置为角标，分别记为 pos0、pos1、pos2，角标位置如下 Figure 7 所示。

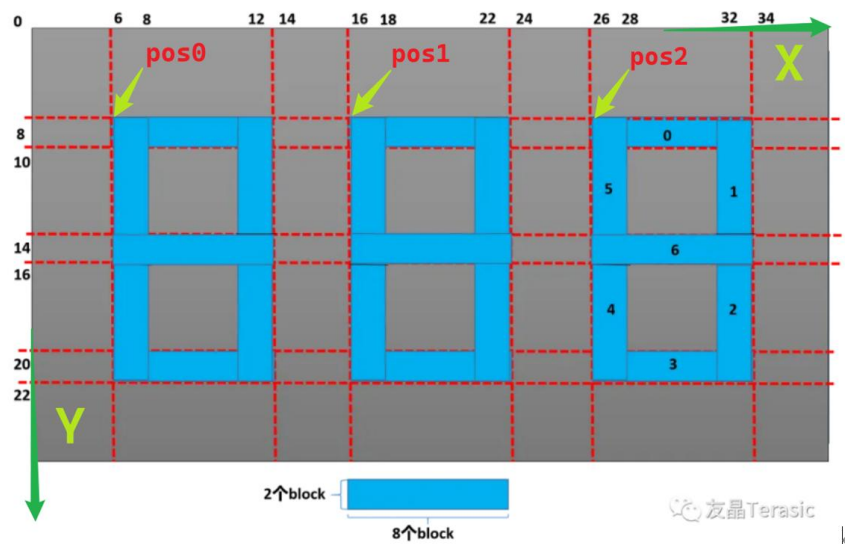


Figure 7 角标位置

对于3位数字的显示情况，有四种情况。

- 三个位置处的数字均未被截断
- pos0 处的数字被截断
- pos1 处的数字被截断
- pos2 处的数字被截断

代码实现中 vga 显示的着色过程便由这四大部分构成。代码总体结构如下 Figure8 所示。

```
always @(posedge clk, posedge reset)
begin
    if (reset)
        vga_rgb <= 12'h000;
    else
        begin
            if (pos0 < (pos0 + 7) % 40 && pos1 < (pos1 + 7) % 40 && pos2 < (pos2 + 7) % 40) //NOTE pos0. pos1. pos2 处的数字均未被截断
                begin...
            else if (pos0 > (pos0 + 7) % 40 && pos1 < (pos1 + 7) % 40 && pos2 < (pos2 + 7) % 40) //NOTE pos0处的数字被截断
                begin...
            else if (pos0 < (pos0 + 7) % 40 && pos1 > (pos1 + 7) % 40 && pos2 < (pos2 + 7) % 40) //NOTE pos1处的数字被截断
                begin...
            else //NOTE pos2处的数字被截断
                begin...
            end
        end
    end
end
```

Figure 8 总体结构

对于第1种情况，3个位置处的数字均未被截断，三个数字的显示区域的Y值是固定的，X的范围在屏幕中都是连续的。显示区域的X坐标有效范围如下 Figure 9 所示。

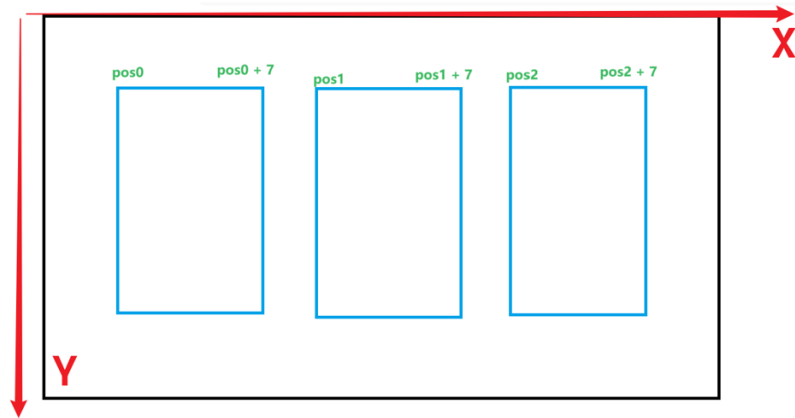


Figure 9 第一种情况的 X 范围

第二种情况，pos0 处的数字被截断。显示区域的 X 坐标有效范围如下 Figure10 所示。其中对于显示区域 1、2、4、5 段的显示处理需要分类讨论，pos0 处的数字被截断的地方可能会出现现在它们的显示区域内，故它们的 X 范围要分类讨论。

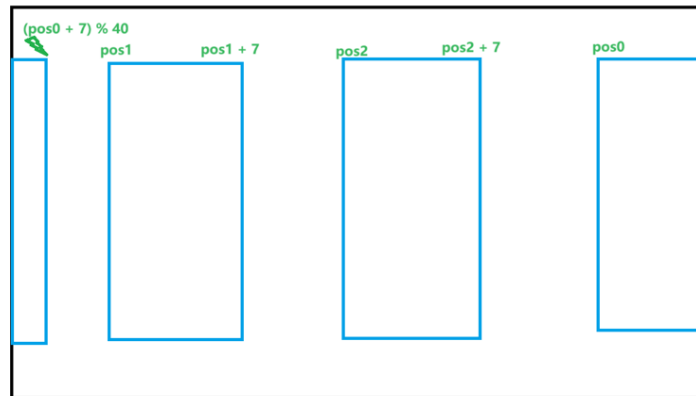


Figure 10 第二种情况的 X 范围

对于剩下的两种情况，和第二种情况的处理相似，只是所要显示的数字和显示位置 posX 变量不同。